



Microservice extraction based on knowledge graph from monolithic applications

Zhiding Li ^a, Chenqi Shang ^a, Jianjie Wu ^a , Yuan Li ^b

[Show more](#)

Share Cite

<https://doi.org/10.1016/j.infsof.2022.106992>

[Get rights and content](#)

Highlights

- Propose a novel method for microservice extraction from monolithic applications;
- Define four types of entities and entity-entity relationships to construct the knowledge graph of the monolithic application, which improves the universality and stability of the method;
- Introduce constraints to the Louvain algorithm for graph clustering to determine microservice candidates;
- Automatically extract information from the existing design resources of the original monolithic application;
- The proposed method can be used as an auxiliary means to check the rationality of the original design of the monolithic application.

Abstract

Context

Re-architecting monolithic systems with microservice architecture is a common trend. However, determining the "optimal" size of individual services during microservice extraction has been a challenge in software engineering. Common limitations of the literature include not being reasonable enough to be put into practical application; relying too much on human experience; neglect of the impact of hardware environment on the performance.

Objective

To address these problems, this paper proposes a novel method based on knowledge-graph to support the extraction of microservices during the initial phases of re-architecting existing applications.

Method

According to the microservice extraction method based on the AKF principle which is a widely practiced microservice design principle in the industry, four kinds of entities and four types of entity-entity relationships are designed and automatically extracted from specification and design artifacts of the monolithic application to build the knowledge graph. A constrained Louvain algorithm is proposed to identify microservice candidates.

Results

Our approach is tested based on two open-source projects with the other three typical methods: the domain-driven design-based method, the similarity calculation-based method, and the graph clustering-based method. Conducted experiments show that our method performs well concerning all the evaluation metrics.

Introduction

The architecture of software systems has changed dramatically in response to the rapid development of the software industry: from monolithic architecture that encapsulates all functions together to vertical architecture that follows the Model-View-Controller (MVC) hierarchy [1], then to Service-Oriented Architecture (SOA) [2]. These designs bring us several advantages, such as simple deployment, minimal labor cost, and easy testing. However, as requirements evolve and business iterates, a monolithic application's code size grows, and the system's complexity rises, posing a slew of issues. New technologies, for example, are hard to integrate into the original design; an increasingly bloated structured program makes deployment and debugging difficult; and the failure of a single module might bring the entire software system down. Researchers have suggested microservice architecture (MSA) [3], which decomposes a massive, complex application system into multiple small-scale, independently deployable services to address the above issues. Each service in a microservice architecture focuses on a single business function and adheres to the idea of single responsibility. In other words, the acquired microservices have their service boundaries and life cycles, and several services collaborate to complete their tasks [4]. The advantages of microservices include strong maintainability, scalability, automated deployment, and flexibility of technology choices. Microservice has been universally acknowledged both in academics and industry. As a result, several of the world's most successful online companies, including Netflix, Amazon, and eBay, have switched to microservice architecture [11][19]. The extraction of microservices, on the other hand, is one of the numerous challenges connected with this procedure. There is no uniform method for determining the boundary and scale of microservices due to various factors such as the complexity of the business logic, the cost of inter-service communications, and the size of the development team [5]. Microservice extraction has attracted considerable attention from scholars [6].

In recent years, some specialists have devoted themselves to research on microservice extraction. Code-scale-based methods [7], domain-driven design [8], similarity-calculation-based algorithms [9], and graph clustering methodologies [10] are all common approaches. However, much of the literature appears to suffer from the following limitations: being difficult to be applied in practical engineering projects due to significant design limitations (code-scale-based methods); leaning too much on human experience, resulting in high time costs (domain-driven design); largely reliant on well-defined interfaces or codes, which many monolithic programs lack (similarity-calculation-based algorithms); seldom focused on hardware environment (graph clustering approaches). This paper presents a novel microservice extraction method to address the mentioned difficulties. First, analyze the system modules, functions, domain entities, and the system's reliance on hardware resources to define four types of entities to construct a knowledge graph. Second, convert the monolithic program into a graph structure that uses these four types of entities. Finally, use a restricted Louvain algorithm to perform graph clustering to get a set of microservice candidates. We apply the proposed method and three other typical microservice extraction methods to two software projects with different scales. The experimental findings

demonstrate that the suggested technique performs well in terms of team size reduction, cohesion, coupling, and code redundancy. The strategy of this study also overcomes some of the limitations of existing methods.

The rest of the paper is arranged as follows. Section2 surveys the related work. Section3 describes the mechanism we proposed for microservice extraction. Section4 presents two typical case studies and discusses the experimental validation. Section5 concludes the work and lists potential future work.

Access through your organization

Check access to the full text by signing in through your organization.

Access through **your organization**

Section snippets

Related Work

With the rise of SOA [11], microservice architectures originated in the agile developer community and have received industry and academic attention in recent years. Microservices, as opposed to monolithic applications, are autonomously deployed services. Microservices have a single, well-defined purpose and provide a variety of benefits [12]. However, migrating to microservices is not an easy task [13,14,15]. One of the difficulties is the problem of microservice extraction. This section will ...

Microservice Extraction Method

Microservice extraction should follow microservice design principles. It may directly benefit from the AKF concept [30], a widely practiced microservice design approach in the industry. The AKF principle asserts that while decomposing a monolithic application, three aspects should be taken into account: data extraction, business function decomposition, and horizontal replication capabilities. Fig.1. shows the AKF Scalability Cube. Restructuring the monolithic application into the three ...

Case Introduction

To validate the proposed algorithm, we assessed it through two case studies.

The first case is an e-commerce project “E-commerce System”, an open-source project developed by Macrozheng et al [42].

The “E-commerce System” uses a layered architecture that supports Web and mobile access. It is built on Spring Boot, a Java-based framework. Use MySQL and MongoDB for data storage. Take Redis for hotspot data storage. ElasticSearch is used for product search and log collection. At present, the scale of ...

Conclusions and Future Work

This paper proposes a novel method based on knowledge-graph to support the extraction of microservices during the initial phases of re-architecting existing applications. According to the microservice extraction method based on the AKF principle, four kinds of entities and four types of entity-entity relationships are designed and automatically extracted from specification and design artifacts of the monolithic application to build the knowledge graph. Propose a constrained Louvain community ...

CRediT authorship contribution statement

Chenqi Shang and Jianjie Wu contributed to the conception of the study;

Zhiding Li verified the method.

Zhiding Li and Chenqi Shang performed the experiments.

Zhiding Li, Chenqi Shang, and Jianjie Wu wrote the manuscript.

Yuan Li, check and review the manuscript.

All authors read and approved the final manuscript. ...

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. ...

[Recommended articles](#)

References (50)

C.H. Chang *et al.*

[Pattern-based framework for modularized software development and evolution robustness](#)

Information and Software Technology (2011)

P. Valderas *et al.*

[A microservice composition approach based on the choreography of BPMN fragments](#)

Information and Software Technology (2020)

J.X. Liu *et al.*

[Service organization and recommendation using multi-granularity approach](#)

Knowledge-Based Systems (2015)

S.S. Li *et al.*

[Understanding and addressing quality attributes of microservices architecture: A Systematic literature review](#)

Information and Software Technology (2021)

J. Soldani *et al.*

[The pains and gains of microservices: a systematic grey literature review](#)

Journal of Systems and Software (2018)

Y.W. Zhang *et al.*

[Service recommendation based on quotient space granularity analysis and covering algorithm on spark](#)

Knowledge-Based Systems (2018)

V. Rampérez *et al.*

[FLAS: A combination of proactive and reactive auto-scaling architecture for distributed services](#)

Future Generation Computer Systems (2021)

D. Ameller *et al.*

Development of service-oriented architectures using model-driven development: A mapping study

Information and Software Technology (2015)

D. Krafzig *et al.*

Enterprise SOA: service-oriented architecture best practices

(2004)

H. Knoche *et al.*

Drivers and barriers for microservice adoption-a survey among professionals in germany

Enterprise Modelling and Information Systems Architectures (EMISA)-International Journal of Conceptual Modeling (2019)



View more references

Cited by (30)

Migration of monolithic systems to microservices: A systematic mapping study

2025, Information and Software Technology

[Show abstract](#) ✓

Cloud-edge microservices architecture and service orchestration: An integral solution for a real-world deployment experience

2023, Internet of Things (Netherlands)

Citation Excerpt :

...The comparison between monolithic and microservices architecture is being discussed by the scientific community [21], as well as the proper management of data inside these Cloud-Edge microservices architectures [22]. Even models to decide whether changing to a microservices architecture or continuing using a monolithic one [23] and methods based on knowledge graphs to the extraction of microservices candidates from monolithic applications [24] are present in the literature. The methods available to manage the orchestration of containers are also under study by using different models to the placement of containers to reduce the deployment cost [25]...

[Show abstract](#) ✓

Microservice extraction using graph deep clustering based on dual view fusion

2023, Information and Software Technology

Citation Excerpt :

...Microservice extraction approaches can be divided into two categories. The first category is based on static analysis, usually using static information such as source code, classes, methods, packages, files, and directories in a monolithic application [3,5–8]. The second category is based on dynamic analysis, usually using dynamic information such as logs, runtime traces, use cases, and runtime data flows. [9–13]...

[Show abstract](#) ✓

Microservices Adoption: An Industrial Inquiry into Factors Influencing Decisions and Implementation Strategies ↗

2024, International Journal of Computing and Digital Systems

Architecture for Self-Evolution of 6G Core Network Based on Intelligent Decision Making ↗

2023, Electronics (Switzerland)

From monolithic to microservice architecture: an automated approach based on graph clustering and combinatorial optimization ↗

2023, Proceedings - IEEE 20th International Conference on Software Architecture, ICSA 2023



View all citing articles on Scopus ↗

[View full text](#)

© 2022 Elsevier B.V. All rights reserved.



All content on this site: Copyright © 2026 Elsevier B.V., its licensors, and contributors. All rights are reserved, including those for text and data mining, AI training, and similar technologies. For all open access content, the relevant licensing terms apply.

